CS6596 — Parallel and concurrent programming (CS6596, CS6868)

Tier	C
Phase (order)	7
Do this first	CS3500, CS2600
Estimated effort	10 weeks
Unlocks	CS6235, CS6886

Roadmap

1. Threads, locks, lock-free ideas; OpenMP/MPI; GPU basics.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Stanford CS149 I Parallel Computing I 2023 I Kayvon Fatahalian and Kunle Olukotun (19 videos). Stanford CS149 *Parallel Computing* (2023, Kayvon Fatahalian & Kunle Olukotun, official): SIMD, multicore, GPUs/CUDA, synchronisation.

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- MIT 6.172 Performance Engineering of Software Systems, Fall 2018 (23 videos). MIT 6.172 *Performance Engineering of Software Systems* (Fall 2018, official OCW): Cilk, caches, races.
- Parallel Computer Architecture (67 videos). NPTEL *Parallel Computer Architecture* (IIT Guwahati).
- Introduction to parallel Programming in Open MP (87 videos). NPTEL *Introduction to Parallel Programming in OpenMP* (IIT Delhi).
- NOC Jan 2020: GPU Architectures and Programming- Prof Soumyajit Dey (64 videos). NPTEL *GPU Architectures and Programming* (Soumyajit Dey, IIT Kharagpur): CUDA.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–3	Why parallelism, multi-core architecture, ISPC. Assignment 1 from the CS149 site (Mandelbrot with threads and SIMD).
2	4–6	Parallel programming basics, work distribution, locality and communication. Assignment 2: task-graph scheduler.

Week	Lectures	What to do
3	7–10	GPU architecture and CUDA, data-parallel thinking, Spark, DNNs on GPUs. Assignment 3: CUDA renderer.
4	11–13	Cache coherence, memory consistency, lock-free data structures. Write a lock-free stack and test it under contention.
5	14–19	DSLs, transactional memory, hardware specialisation, memory access.

▼ **All lectures in *Stanford CS149 I Parallel Computing I 2023 I Kayvon Fatahalian and Kunle Olukotun* (19 videos)**

1. Stanford CS149 I Parallel Computing I 2023 I Lecture 1 - Why Parallelism? Why Efficiency?
2. Stanford CS149 I Parallel Computing I 2023 I Lecture 2 - A Modern Multi-Core Processor
3. Stanford CS149 I 2023 I Lecture 3 - Multi-core Arch Part II + ISPC Programming Abstractions
4. Stanford CS149 I Parallel Computing I 2023 I Lecture 4 - Parallel Programming Basics
5. Stanford CS149 I 2023 I Lecture 5 - Performance Optimization I: Work Distribution and Scheduling
6. Stanford CS149 I Lecture 6 - Performance Optimization II: Locality, Communication, and Contention
7. Stanford CS149 I Parallel Computing I 2023 I Lecture 7 - GPU architecture and CUDA Programming
8. Stanford CS149 I Parallel Computing I 2023 I Lecture 8 - Data-Parallel Thinking
9. Stanford CS149 I 2023 I Lecture 9 - Distributed Data-Parallel Computing Using Spark
10. Stanford CS149 I Parallel Computing I 2023 I Lecture 10 - Efficiently Evaluating DNNs on GPUs
11. Stanford CS149 I Parallel Computing I 2023 I Lecture 11 - Cache Coherence
12. Stanford CS149 I Parallel Computing I 2023 I Lecture 12 - Memory Consistency
13. Stanford CS149 I 2023 I Lecture 13 - Fine-Grained Synchronization and Lock-Free Programming
14. Stanford CS149 I Parallel Computing I 2023 I Lecture 14 - Midterm Review
15. Stanford CS149 I Parallel Computing I 2023 I Lecture 15 - Domain Specific Programming Languages
16. Stanford CS149 I Parallel Computing I 2023 I Lecture 16 - Transactional Memory 1
17. Stanford CS149 I Parallel Computing I 2023 I Lecture 17 - Transactional Memory 2
18. Stanford CS149 I Parallel Computing I 2023 I Lecture 18 - Hardware Specialization
19. Stanford CS149 I Parallel Computing I 2023 I Lecture 19 - Accessing Memory + Course Wrap Up

Books, notes and websites

- Book: Herlihy & Shavit, *The Art of Multiprocessor Programming*
- LLNL OpenMP and MPI tutorials
- CUDA programming guide

Practice / proof of understanding

Parallelise matrix multiply with OpenMP and measure speedup.

What your IITM course sheet says

From the PDF: CS6596

Official description: It will cover parallel programming for shared-memory multiprocessors, distributed memory systems and GPUs, as well as the fundamental performance issues with these systems. It would help the attendees learn the issues involved in creating high-performance parallel software. The course is intended to be of interest to both computer scientists as well as computational scientists who use high-performance computing in their work.

Official syllabus (trimmed):

> Caches and locality: Review of cache structure; cache mapping of address tracesLoop transformations: Loop permutation, unrolling, tiling, fusionData dependence analysis: Dependence vectors, validity of loop transformationsPerformance modeling: Modeling cache miss rate; transformations to reduce missesOpenMP review: Parallel regions; work sharing; synchronization; data storage attributesOpenMP performance optimization: False sharing; load balancing; minimizing synchronization; performance toolsOpenMP case studies: LU Decomposition; Finite Difference Time Domain (FDTD)GPU Programming: Computation hierarchy; Memory hierarchy; SIMD; CUDA, OpenCL; SynchronizationGPU performance issues: Thread divergence; Memory coalescing; Bank conflicts; Reductions, Prefix-sums; Memoryoptimizations; Zero-copy; Overlapping communication and computationGPU case studies: Dense matrices; shared worklist; image filtersIrregular algorithms: Sparse matrices; graph algorithmsVectorization: SSE/AVX vector ISA; Dependence constraints forIntel Xeon Phi case studies: Rank-k update; Multigrid smootherDistributed Computing with MPI (3 weeks): Communicator & Communication - point to point & collective function calls, Examples such as leader election and counting, Performance: communication volume, bandwidth, Case study: Domain decomposition

Official textbooks: Parallel computing: Principles and practice; T. J. Fountain; Cambridge University Press; 2009Using OpenMP: Portable shared memory parallel programming; Barbara Chapman, Gabriele Jost and Ruud van der Pas; MIT Press; 2007

Official references: Measuring computer performance: A practitioner's guide; David J. Lilja; Cambridge University Press; 2009Relevant conference and journal papers.

From the PDF: CS6868

Official description: The course will approach concurrent programming from two directions. In the first part of the course, the abstractions used in concurrent programming are explored. This part will build the student's intuition on understanding and reasoning about concurrency. The second part of the course will focus on performance. It is imperative for programmers who write concurrent code to understand the fundamentals and the limitations of the underlying architectures. Different implementation models of concur

Official syllabus (trimmed):

> Introduction: Shared objects and synchronization; parallel programming; Threads and monitors, sleeping; Interconnect, memory, caches, cache-conscious programming; Multicore and multithreaded architectures, hardware synchronization instructions. Abstractions: 1. Mutual exclusion: solutions to two- thread problems; locks, fairness and timestamps; 2. Concurrent objects: different forms of consistency, linearizability, progress and memory models of modern programming languages; 3. Foundations of shared memory, correctness arguments; 4. Synchronization primitives including consensus, atomic registers, compare_and_set; 5. Lock-free and wait-free construction, universality of consensus. Implementation: 1. Spin locks and contention, test-and-set locks, exponential backoff, composite and hierarchical locks; 2. Monitors – locks and conditions, readers-writers locks; reentrant locks, semaphores. Concurrent Data Structures and Algorithms: 1. Linked lists – fine grained and coarse grained synchronization; optimistic and lazy synchronization; 2. Concurrent queues and the ABA problem; concurrent stacks and elimination backoff, rendezvous points; 3. Counting, sorting and distributed co-ordination; 4. Concurrent hashing, search, priority queues; 5. Barriers, transactional memory; 5. Debugging concurrent programs, exception handling.

Official textbooks: 1. “The Art of Multiprocessor Programming” by Maurice Herlihy and NirShavit, Morgan Kaufmman Publishers, 1st Edition, Indian Reprint 2012. 2. “The Art of Concurrency: A Thread Monkey's Guide to Writing Parallel Applications” by O' Reilly Media, 1st Edition, 2009

Official references: 1. “Java Concurrency in Practice” by Brian Goetz, Tim Peierls, Joshua Block, Joseph Bowbeer, David Holmes and Doug Lea, Addison Wesley, 1st Edition, 2006 2. “An Introduction to Parallel Programming” by Peter Pacheco, Morgan Kaufmann, 1st Edition, 2011 3. Selected reading from papers published in leading conferences and journals.